

Lab6：进程调度

南开大学 计算机学院和密码与网络空间安全学院

年级：2023 级

专业：信息安全

姓名及学号：于昊汐（2312642）、陈安琪（2312481）、孙俪菲（2312724）

日期：2026 年 1 月 2 日

一、实验目的

1. 理解操作系统进程调度的核心机制与调度器框架设计思想；
2. 熟悉 ucore 调度器框架的解耦逻辑，实现缺省的 Round-Robin（时间片轮转）调度算法；
3. 基于该框架完成 Stride 调度算法，掌握不同调度策略的实现思路与适用场景。

二、实验内容

- 练习 0：整合实验 2-5 的相关代码至本实验对应标注位置，更新进程控制块调度信息初始化及时钟中断处理逻辑，确保编译通过；
- 练习 1：分析 ucore 调度器框架核心组件（sched_class、run_queue 结构体），梳理调度类初始化、进程调度流程及算法切换机制，理解调度框架与具体算法的解耦设计；
- 练习 2：基于调度器框架，实现 Round-Robin 调度算法的 init、enqueue、dequeue、pick_next、proc_tick 核心函数，验证算法正确性并分析其性能；
- Challenge1：切换调度算法为 Stride Scheduling，利用斜堆数据结构实现该算法的核心接口，验证优先级与 CPU 分配的比例关系，完成算法设计与实现。

三、实验过程

练习0：填写已有实验

本实验依赖实验2/3/4/5。请把你做的实验2/3/4/5的代码填入本实验中代码中有“LAB2”/“LAB3”/“LAB4”/“LAB5”的注释相应部分。并确保编译通过。注意：为了能够正确执行lab6的测试应用程序，可能需对已完成的实验2/3/4/5的代码进行进一步改进。由于我们在进程控制块中记录了一些和调度有关的信息，例如Stride、优先级、时间片等等，因此我们需要对进程控制块的初始化进行更新，将调度有关的信息初始化。同时，由于时间片轮转的调度算法依赖于时钟中断，你可能也要对时钟中断的处理进行一定的更新。

ANS:

主要对比lab5，这次实验在填写之前实验空缺时，做出了以下这些变化，主要在这几个文件上：

trap.c

仅修改了时钟中断处理（IRQ_S_TIMER分支）部分，删除了Lab5的硬编码逻辑，新增调度器框架接口调用：

```
// Lab6 IRQ_S_TIMER 分支代码
case IRQ_S_TIMER:
    // 保留 Lab3 基础逻辑：设置下次时钟中断、 ticks 计数
    clock_set_next_event();
    ticks++;

    // Lab6 新增代码：调用调度器框架的 proc_tick 接口
    if (current != NULL)
    {
        sched_class_proc_tick(current);
    }
    break;
```

为啥这次改成这样，核心原因是为了适配调度器框架与算法解耦的设计。

Lab6的核心目标是让ucore支持多种调度算法（RR/Stride/自定义），且切换算法时无需修改框架代码。因此 Lab6 废弃了硬编码，转而通过 调度器框架接口 对接具体算法，sched_class_proc_tick 是 sched_class 结构体的函数指针，指向当前激活的调度类（如 RR/Stride）的 proc_tick 实现。

那么为什么依旧保留 clock_set_next_event() 和 ticks++。首先，clock_set_next_event()是时钟硬件的基础配置，用于设置下一次时钟中断的触发时间，确保时钟中断能周期性发生；ticks++是全局时钟计数器，用于记录系统运行时间用于调试，与调度算法解耦，保留不影响框架。

proc.c

新增调度相关字段初始化（alloc_proc 函数中）：

```
/*
 * below fields(add in LAB6) in proc_struct need to be initialized
 *      struct run_queue *rq;                                // run queue contains
Process
 *      list_entry_t run_link;                                // the entry linked in run
queue
 *      int time_slice;                                       // time slice for occupying
the CPU
 *      skew_heap_entry_t lab6_run_pool;                     // entry in the run pool
(lab6 stride)
 *      uint32_t lab6_stride;                                 // stride value (lab6
stride)
 *      uint32_t lab6_priority;                               // priority value (lab6
stride)
 */
// Initialize LAB6 additions
proc->rq = NULL; // 关联的运行队列（初始为空）
list_init(&(proc->run_link)); // 初始化就绪队列链表节点
proc->time_slice = 0; // 时间片（后续由调度算法初始化）
skew_heap_init(&(proc->lab6_run_pool)); // 初始化Stride算法的优先队列节点
proc->lab6_stride = 0; // Stride值（初始为0）
proc->lab6_priority = 1; // 优先级（默认1，值越大优先级越高）
```

新增进程优先级设置函数：

```
// FOR LAB6, set the process's priority (bigger value will get more CPU time)
void lab6_set_priority(uint32_t priority)
{
    cprintf("set priority to %d\n", priority);
    if (priority == 0)
        current->lab6_priority = 1; // 优先级最小为1, 避免除零错误
    else
        current->lab6_priority = priority;
}
```

对 proc.c 的改动核心是为调度算法提供进程级元数据支持：

通过新增 proc_struct 调度字段，让进程能适配 RR/Stride 两种不同算法的就绪队列结构和逻辑；

通过 lab6_set_priority 函数支持优先级配置，实现按优先级分配CPU时间的调度目标；

所有改动都与 Lab6 的调度器框架接口对齐，确保调度算法能通过框架接口操作进程的调度字段，最终实现框架不变，算法可动态切换的核心设计思想。

练习1: 理解调度器框架的实现（不需要编码）

请仔细阅读和分析调度器框架的相关代码，特别是以下两个关键部分的实现：

在完成练习0后，请仔细阅读并分析以下调度器框架的实现：

调度类结构体 sched_class 的分析：请详细解释 sched_class 结构体中每个函数指针的作用和调用时机，分析为什么需要将这些函数定义为函数指针，而不是直接实现函数。

运行队列结构体 run_queue 的分析：比较lab5和lab6中 run_queue 结构体的差异，解释为什么lab6的 run_queue 需要支持两种数据结构（链表和斜堆）。

调度器框架函数分析：分析 sched_init()、wakeup_proc() 和 schedule() 函数在lab6中的实现变化，理解这些函数如何与具体的调度算法解耦。

对于调度器框架的使用流程，请在实验报告中完成以下分析：

调度类的初始化流程：描述从内核启动到调度器初始化完成的完整流程，分析 default_sched_class 如何与调度器框架关联。

进程调度流程：绘制一个完整的进程调度流程图，包括：时钟中断触发、proc_tick 被调用、schedule() 函数执行、调度类各个函数的调用顺序。并解释 need_resched 标志位在调度过程中的作用

调度算法的切换机制：分析如果要添加一个新的调度算法（如stride），需要修改哪些代码？并解释为什么当前的设计使得切换调度算法变得容易。

ANS:

问题一

sched_class 结构体中每个函数指针的作用与调用时机：

struct sched_class 是 ucore 调度器框架的核心，每个函数指针对应调度算法的一个通用接口，具体实现由 RR/Stride 等算法完成，框架通过调用这些接口实现调度逻辑。

1. void (*init)(struct run_queue *rq);

- 作用：初始化运行队列（run_queue），为调度算法准备数据结构（如链表头、斜堆、进程计数等）。
- 调用时机：仅在系统启动时调用一次：

```
// sched.c 中 sched_init 片段
rq = &__rq;
rq->max_time_slice = MAX_TIME_SLICE;
sched_class->init(rq); // 调用当前调度类的 init 接口
```

2. void (*enqueue)(struct run_queue *rq, struct proc_struct *proc);

- 作用：将就绪态进程加入运行队列，维护就绪队列的结构。
- 调用时机：进程从非就绪态变为就绪态时触发，核心场景有 2 个：
 - 进程被唤醒（wakeup_proc 函数）：进程从睡眠态（PROC_SLEEPING）唤醒为就绪态（PROC_RUNNABLE），调用 sched_class_enqueue(proc) 入队；
 - 调度切换时（schedule 函数）：当前运行进程仍为就绪态（如时间片未耗尽但被抢占），将其重新入队，等待下次调度。
- 核心要求：入队时需保证运行队列的完整性，并更新 rq->proc_num（进程计数）。

3. void (*dequeue)(struct run_queue *rq, struct proc_struct *proc);

- 作用：将进程从运行队列中移除。
- 调用时机：进程脱离就绪队列时触发，核心场景有 2 个：
 - 进程被选中执行（schedule 函数）：调度器通过 pick_next 选中下一个进程后，调用 sched_class_dequeue(next) 将其从就绪队列移除；
 - 进程退出 / 睡眠（如 do_exit、do_sleep）：进程状态从 PROC_RUNNABLE 变为 PROC_ZOMBIE 或 PROC_SLEEPING，需从就绪队列中删除。
- 核心要求：出队时需正确维护就绪队列结构，并更新 rq->proc_num。

4. struct proc_struct (pick_next)(struct run_queue *rq);

- 作用：从运行队列中选择下一个要执行的进程，是调度算法的核心逻辑。
- 调用时机：每次调度切换时，在当前进程入队后调用，获取下一个执行的进程。
- 边界处理：若就绪队列为空，无用户进程，返回 NULL，调度器会自动选择 idleproc。

5. void (*proc_tick)(struct run_queue *rq, struct proc_struct *proc);

- 作用：时钟中断触发时，更新当前运行进程的调度状态，如时间片递减、判断是否需要调度。
- 调用时机：每一次时钟中断（IRQ_S_TIMER）处理时，通过 sched_class_proc_tick(current) 调用：

```
if (current != NULL)
{
    sched_class_proc_tick(current); // 调用当前调度类的 proc_tick 接口
}
```

为什么要用函数指针而非直接实现函数？

核心原因是实现调度框架与算法解耦，让 ucore 支持动态切换调度算法，同时降低新增算法的开发成本。

1. 解耦框架与算法，避免硬编码

- 如果直接在框架中实现调度逻辑，会导致框架与算法强耦合，修改算法需改动 sched.c 核心代码，风险高；无法支持多算法：要切换 RR/Stride，需删除旧逻辑、添加新逻辑，扩展性差。
- 而函数指针将调度算法的实现与框架的调用逻辑分离，框架只负责何时调用接口，不关心接口如何实现；
- 算法只负责实现接口逻辑，不关心框架何时调用。

2. 支持动态切换调度算法

- 通过修改 sched_init 中 sched_class 的赋值，即可切换调度算法，无需改动框架其他代码：

```
// 切换为 RR 算法
sched_class = &default_sched_class;
// 切换为 Stride 算法
sched_class = &stride_sched_class;
```

- 这种设计类似面向对象的多态，sched_class 是一个抽象基类，RR/Stride 是具体子类，框架通过统一接口调用不同子类的实现。

3. 降低新增算法的开发成本

- 新增调度算法时，只需：
 - 实现 sched_class 的 5 个函数指针 (init/enqueue/dequeue/pick_next/proc_tick) ；
 - 定义一个 struct sched_class 实例，将函数指针指向自定义实现；
 - 在 sched_init 中切换为新的调度类。
- 无需修改框架的核心逻辑，符合 开闭原则。

4. 统一接口，简化框架逻辑

- 无论算法的内部实现多么复杂，框架都通过统一的 5 个接口与算法交互，避免了框架中出现大量条件判断，让框架代码更简洁、易维护。

总结来说，每个函数指针对应调度流程的一个核心步骤，明确了何时调用 和做什么，实现了算法的标准化；而函数指针的间接调用实现了 框架与算法解耦，支持动态切换算法、降低扩展成本，是 面向接口编程 在 C 语言中的经典应用。

问题二

Lab5 与 Lab6 中 调度队列 的核心差异

首先明确，Lab5 并未单独定义 run_queue 结构体，因为 FIFO 算法足够简单，无需封装独立的运行队列，而是直接复用全局进程链表 proc_list 实现调度；Lab6 则封装了 run_queue 结构体，支持双数据结构。两者的核心差异如下：

1. Lab5 的 调度队列 实现（无独立 run_queue 结构体）

- Lab5 仅实现 FIFO 调度，核心依赖 全局进程链表 proc_list，所有进程都在该链表中，调度逻辑直接操作该链表：
 - 核心字段：无独立 run_queue，仅依赖全局变量 proc_list；
 - 就绪进程管理：通过遍历 proc_list 筛选 state == PROC_RUNNABLE 的进程；
 - 调度逻辑（schedule 函数）：
 - 从当前进程的链表节点开始遍历 proc_list；
 - 找到第一个状态为 PROC_RUNNABLE 的进程作为下一个执行进程；
 - 若未找到，则选择 idleproc。

2. Lab6 的 run_queue 结构体（独立封装 + 双数据结构）

- Lab6 为支持多调度算法，封装了独立的 run_queue 结构体，定义在 sched.h 中，核心字段如下：

```
struct run_queue {
    list_entry_t run_list;           // 链表：服务于 RR 算法
    unsigned int proc_num;           // 就绪进程数（通用字段）
    int max_time_slice;              // 最大时间片（RR 算法核心）
    skew_heap_entry_t *lab6_run_pool; // 斜堆：服务于 Stride 算法
};
```

- 特点如下：
 - 独立于全局进程链表，run_queue 仅维护 就绪态 进程，与 proc_list 解耦；
 - 双数据结构：链表（run_list）和斜堆（lab6_run_pool）按需使用，分别适配 RR 和 Stride 算法。

对比维度	Lab5 实现	Lab6 实现
调度队列形态	无独立 run_queue，复用全局进程链表 proc_list	独立 run_queue 结构体，封装就绪进程
数据结构	仅全局链表（proc_list）	双数据结构：链表（run_list）+ 斜堆（lab6_run_pool）
核心字段	无额外字段，依赖进程的 state 和 list_link	包含链表头、进程数、最大时间片、斜堆头指针
就绪进程筛选	遍历 proc_list，判断 state == PROC_RUNNABLE	就绪进程已提前加入 run_queue，无需全局遍历
支持算法	仅 FIFO 算法	RR 算法（链表）、Stride 算法（斜堆）

也就是说，Lab5 的 调度队列 是无状态遍历，每次调度都要遍历全局进程链表，筛选就绪进程，效率低但实现简单；Lab6 的 run_queue 是有状态管理，提前将就绪进程加入专门的队列（链表 / 斜堆），调度时直接操作队列，效率高且支持多算法。

Lab6 的 run_queue 支持 链表 + 斜堆 的核心原因：

- 链表（run_list）：适配 RR 算法的 公平轮转 需求
 - RR 算法的核心逻辑是 为每个进程分配固定时间片，时间片耗尽则轮转，其对调度队列的需求是 先进先出、高效入队 / 出队：
 - 入队（enqueue）：进程时间片耗尽或被抢占时，插入 run_list 尾部，保证 轮转公平性，操作复杂度 O(1)；
 - 出队（dequeue）：进程被选中执行时，从 run_list 头部删除，操作复杂度 O(1)；
 - 选择下一个进程（pick_next）：直接取 run_list 头部，复杂度 O(1)；
 - 时间片管理：run_queue 的 max_time_slice 字段为 RR 算法提供统一的时间片配置，解决 Lab5 无时间片的问题。
 - 若用斜堆实现 RR 算法，会导致入队 / 出队复杂度从 O(1) 上升到 O(log n)，完全没必要，链表是 RR 算法的 最优数据结构。
- 斜堆（lab6_run_pool）：适配 Stride 算法的 优先级调度 需求
 - Stride 算法的核心逻辑是 按优先级分配 CPU 时间，其对调度队列的核心需求是 快速查找最小步长进程，步长越小，优先级越高，越先被调度）：
 - 若用链表实现 Stride 算法，pick_next 需遍历整个链表查找最小步长进程，复杂度 O(n)，进程数多时有明显性能瓶颈；
 - 斜堆的优势：斜堆是高效的优先队列，支持 O(log n) 复杂度的 插入、删除、查找最小值操作：
 - 入队（enqueue）：将进程的步长节点插入斜堆，自动维护堆序，最小步长在堆顶；
 - 选择下一个进程（pick_next）：直接取堆顶节点，复杂度 O(1)；
 - 出队（dequeue）：从斜堆中删除堆顶节点，调整堆结构，复杂度 O(log n)。
- 双数据结构共存，解耦算法与框架，避免冗余
 - Lab6 的 run_queue 并非 同时使用两种数据结构，而是算法按需选择：

- 启用 RR 算法时：仅使用 run_list（链表）和 max_time_slice 字段，lab6_run_pool 为 NULL；
- 启用 Stride 算法时：仅使用 lab6_run_pool（斜堆）和 proc_num 字段，run_list 未被使用；
- 通用字段 proc_num 同时服务于两种算法，记录就绪进程数，避免重复定义。
- 这种设计的优势：
 - 算法独立：RR 和 Stride 算法的实现互不干扰，修改一种算法不会影响另一种；
 - 框架统一：两种算法都通过 sched_class 接口与 run_queue 交互，框架无需修改即可切换算法；
 - 效率最优：每种算法都使用最适合自己的数据结构，避免用一种数据结构适配所有算法的效率损耗。

所以，Lab6 的 run_queue 提前将就绪进程与其他状态进程分离，调度时直接操作就绪队列，复杂度从 $O(n)$ 降至 $O(1)$ (RR) 或 $O(\log n)$ (Stride)，同时通过双数据结构支持多算法，扩展性大幅提升。

问题三

三个核心函数的实现变化与解耦分析

1. sched_init(): 从无到调度类初始化

- (1) Lab5 中：无 sched_init() 函数
 - 无需初始化调度算法，FIFO 逻辑简单，无额外数据结构，系统启动后直接通过 schedule() 遍历全局链表调度，无初始化步骤。
- (2) Lab6 中：sched_init() 实现：

```
void sched_init(void)
{
    list_init(&timer_list);

    // 切换调度类（仅需修改这一行，即可切换 RR/Stride 算法）
    sched_class = &stride_sched_class; // 或 &default_sched_class (RR)

    rq = &__rq;
    rq->max_time_slice = MAX_TIME_SLICE;
    sched_class->init(rq); // 调用当前调度类的 init 接口

    cprintf("sched class: %s\n", sched_class->name);
}
```

- 变化与解耦逻辑：
 - 新增核心逻辑：
 - 选择调度类：通过赋值不同的调度类实例，切换算法；
 - 初始化运行队列：调用调度类的 init 接口，初始化该算法所需的数据结构。
 - 解耦体现：
 - 框架不关心如何初始化运行队列：RR 算法初始化链表，Stride 算法初始化斜堆，均由调度类的 init 接口实现；
 - 切换算法仅需修改调度类赋值：无需改动 sched_init() 框架逻辑，实现 算法热切换。

2. wakeup_proc(): 从 仅改状态 到 状态 + 入队接口

- (1) Lab5 中 wakeup_proc() 的实现，仅将进程状态从 PROC_SLEEPING 改为 PROC_RUNNABLE，无 加入就绪队列 步骤，因为 Lab5 无独立就绪队列，调度时实时遍历全局链表。若后续修改算法，需在该函数中添加 加入链表逻辑，改动框架核心代码。

- (2) Lab6 中 wakeup_proc() 实现:

```
void wakeup_proc(struct proc_struct *proc)
{
    assert(proc->state != PROC_ZOMBIE);
    bool intr_flag;
    local_intr_save(intr_flag);
    {
        if (proc->state != PROC_RUNNABLE)
        {
            proc->state = PROC_RUNNABLE;
            proc->wait_state = 0;
            if (proc != current)
            {
                sched_class_enqueue(proc); // 调用调度类的 enqueue 接口
            }
        }
        else
        {
            warn("wakeup runnable process.\n");
        }
    }
    local_intr_restore(intr_flag);
}
```

- 新增核心逻辑 sched_class_enqueue(proc), 间接调用 sched_class->enqueue(rq, proc)。
- 解耦体现:
 - 框架不关心 如何将进程加入就绪队列, RR 算法将进程加入链表, Stride 算法将进程加入斜堆, 均由调度类的 enqueue 接口实现;
 - 函数仅负责 修改状态 + 调用接口, 与具体入队逻辑完全分离。

3. schedule(): 从 内嵌 FIFO 逻辑 到 调用调度类接口

- 这是调度框架的核心, 变化最大, 解耦逻辑最明显。
- Lab5 中, schedule() 内嵌 FIFO 算法, 遍历全局链表 proc_list, 找到第一个就绪进程, 直接切换。
- 这样会面临耦合问题:
 - 算法逻辑与框架强绑定: 修改算法需重写整个 schedule() 函数;
 - 无独立就绪队列: 每次调度都要遍历所有进程, 效率低。
- Lab6 中 schedule() 的实现:

```
void schedule(void)
{
    bool intr_flag;
    struct proc_struct *next;
    local_intr_save(intr_flag);
    {
        current->need_resched = 0;
        if (current->state == PROC_RUNNABLE)
        {
            sched_class_enqueue(current); // 1. 调用 enqueue 接口: 当前进程入队
        }
        if ((next = sched_class_pick_next()) != NULL)
        {
            sched_class_dequeue(next); // 2. 调用 dequeue 接口: 下一个进程出队
        }
    }
}
```

```

    }
    if (next == NULL)
    {
        next = idleproc;
    }
    next->runs++;
    if (next != current)
    {
        proc_run(next); // 3. 进程切换
    }
}
local_intr_restore(intr_flag);
}

```

- 核心逻辑仅包含 调度流程框架，具体算法逻辑通过 3 个接口调用实现：
 - sched_class_enqueue(current): 当前进程若仍为就绪态，加入就绪队列；
 - sched_class_pick_next(): 选择下一个要执行的进程；
 - sched_class_dequeue(next): 将选中的进程从就绪队列中移除。
- 这样就真正实现了解耦：
 - 框架与算法完全分离：schedule() 仅定义 入队→选进程→出队→切换 的流程，不关心队列是链表还是斜堆、如何选进程；
 - 算法细节封装在调度类中：修改算法只需修改调度类的 enqueue/pick_next/dequeue 接口，无需改动 schedule() 框架；
 - 效率提升：就绪进程已提前加入 run_queue，无需遍历全局链表，筛选就绪进程的逻辑由调度类高效实现。

问题四

调度类的初始化完整流程（以 RR 算法为例）

步骤 1：内核启动入口（kern/init/init.c 的 kern_init 函数）

kern_init 是 ucore 内核的启动总入口，所有核心模块的初始化都在这里触发，调度器初始化是其中关键一步：

```

int kern_init(void) {
    // ... 其他初始化（控制台、物理内存、中断、虚拟内存等）
    sched_init(); // 第 1 步：调用调度器框架初始化函数
    proc_init();  // 第 2 步：初始化进程管理（创建 idleproc/initproc）
    clock_init(); // 初始化时钟中断（为调度提供时间触发条件）
    intr_enable(); // 启用中断（时钟中断才能正常触发）
    cpu_idle();   // 运行空闲进程，进入调度循环
}

```

步骤 2：调度器框架初始化（kern/schedule/sched.c 的 sched_init 函数）

sched_init 是调度器框架的初始化入口，负责绑定调度类、初始化运行队列，是框架与算法关联的核心步骤：

```

void sched_init(void) {
    list_init(&timer_list); // 初始化定时器链表

    // 第 3 步：绑定调度类（选择 RR 算法，关键关联点）
    sched_class = &default_sched_class; // 全局指针指向 RR 调度类实例

    // 第 4 步：初始化全局运行队列 `rq`
    rq = &__rq; // 全局运行队列实例（`__rq` 是静态全局变量）
}

```

```

rq->max_time_slice = MAX_TIME_SLICE; // 设置 RR 算法的最大时间片

// 第 5 步：调用调度类的 `init` 接口，初始化算法专属数据结构
sched_class->init(rq); // 等价于 default_sched_class.init(rq) → 调用 RR_init

printf("sched class: %s\n", sched_class->name); // 打印当前调度类
}

```

步骤 3：调度类的 init 接口调用（default_sched.c 的 RR_init 函数）

RR_init 是 RR 算法对 sched_class 接口的具体实现，负责初始化 RR 算法专属的就绪队列：

```

static void RR_init(struct run_queue *rq) {
    list_init(&rq->run_list); // 第 6 步：初始化 RR 算法的链表就绪队列
    rq->proc_num = 0; // 就绪进程数初始化为 0
    rq->lab6_run_pool = NULL; // Stride 算法的斜堆字段，RR 算法无需使用，设为 NULL
}

```

步骤 4：进程管理初始化（kern/process/proc.c 的 proc_init 函数）

proc_init 负责创建系统的前两个进程（idleproc 空闲进程、initproc 初始化进程），这些进程是调度的对象，调度器初始化后需要有进程可调度：

```

void proc_init(void) {
    // ... 初始化进程链表、哈希表
    idleproc = alloc_proc(); // 创建空闲进程（PID=0）
    idleproc->state = PROC_RUNNABLE; // 设为就绪态
    // ... 创建 initproc（PID=1，后续会创建用户进程）
    current = idleproc; // 当前进程设为 idleproc
}

```

步骤 5：调度器就绪，进入调度循环（cpu_idle 函数）

cpu_idle 是空闲进程的入口函数，本质是一个调度循环，触发调度器开始工作：

```

void cpu_idle(void) {
    while (1) {
        if (current->need_resched) { // 检测是否需要调度（时钟中断触发）
            schedule(); // 调用调度器核心函数，选择下一个进程
        }
    }
}

```

启动调度循环，当 need_resched 被置位（时钟中断触发 RR_proc_tick），调用 schedule() 执行调度；此时 default_sched_class 已绑定，run_queue 已初始化，进程已创建，时钟中断已启用，调度器具备可调度的所有条件。

default_sched_class 与调度器框架的关联逻辑

default_sched_class 是 RR 算法的调度类实例，它与调度器框架的关联基于接口实现 + 指针绑定，完全遵循面向接口编程思想，具体关联方式如下：

1. 关联基础：sched_class 接口规范（sched.h）

struct sched_class 是调度器框架定义的抽象接口，所有调度算法（RR/Stride）都必须实现该接口的 5 个函数指针，这是框架能调用任意算法的前提：

```

struct sched_class {
    const char *name;                // 调度类名称
    void (*init)(struct run_queue *rq); // 初始化运行队列
    void (*enqueue)(struct run_queue *rq, struct proc_struct *proc); // 进程入队
    void (*dequeue)(struct run_queue *rq, struct proc_struct *proc); // 进程出队
    struct proc_struct *(*pick_next)(struct run_queue *rq); // 选择下一个进程
    void (*proc_tick)(struct run_queue *rq, struct proc_struct *proc); // 时钟中断回调
};

```

2. 关联核心: default_sched_class 实现接口

default_sched_class 是 RR 算法的接口实现实例，它是一个 struct sched_class 类型的全局变量，将自身的 RR 算法函数绑定到接口的函数指针上：

```

// RR 算法的具体实现函数
static void RR_init(...) { ... }
static void RR_enqueue(...) { ... }
static void RR_dequeue(...) { ... }
static struct proc_struct *RR_pick_next(...) { ... }
static void RR_proc_tick(...) { ... }

// 第 1 层关联: RR 算法函数 → sched_class 接口
struct sched_class default_sched_class = {
    .name = "RR_scheduler", // 调度类名称
    .init = RR_init,        // 接口函数指针 → RR 算法实现
    .enqueue = RR_enqueue,
    .dequeue = RR_dequeue,
    .pick_next = RR_pick_next,
    .proc_tick = RR_proc_tick,
};

```

3. 关联触发: 框架通过全局指针调用接口

调度器框架定义了全局指针 sched_class (struct sched_class *sched_class)，通过该指针间接调用 default_sched_class 的接口，实现框架对算法的无感知调用：

- 绑定: sched_init 中 sched_class = &default_sched_class，全局指针指向 RR 调度类实例；
- 调用示例 1: 初始化运行队列 → sched_class->init(rq) → 等价于 RR_init(rq)；
- 调用示例 2: 进程入队 → sched_class->enqueue(rq, proc) → 等价于 RR_enqueue(rq, proc)；
- 调用示例 3: 时钟中断回调 → sched_class->proc_tick(rq, proc) → 等价于 RR_proc_tick(rq, proc)。

4. 关联结果: RR 算法就绪，可响应调度请求

初始化完成后，框架与 default_sched_class 形成稳定关联，后续调度流程都会通过框架调用 RR 算法的接口：

- 进程唤醒 (wakeup_proc)：调用 sched_class->enqueue → 进程加入 run_list 链表；
- 时钟中断 (trap.c)：调用 sched_class->proc_tick → 进程时间片递减，耗尽则置位 need_resched；
- 调度切换 (schedule)：调用 sched_class->pick_next → 选择链表头部进程执行。

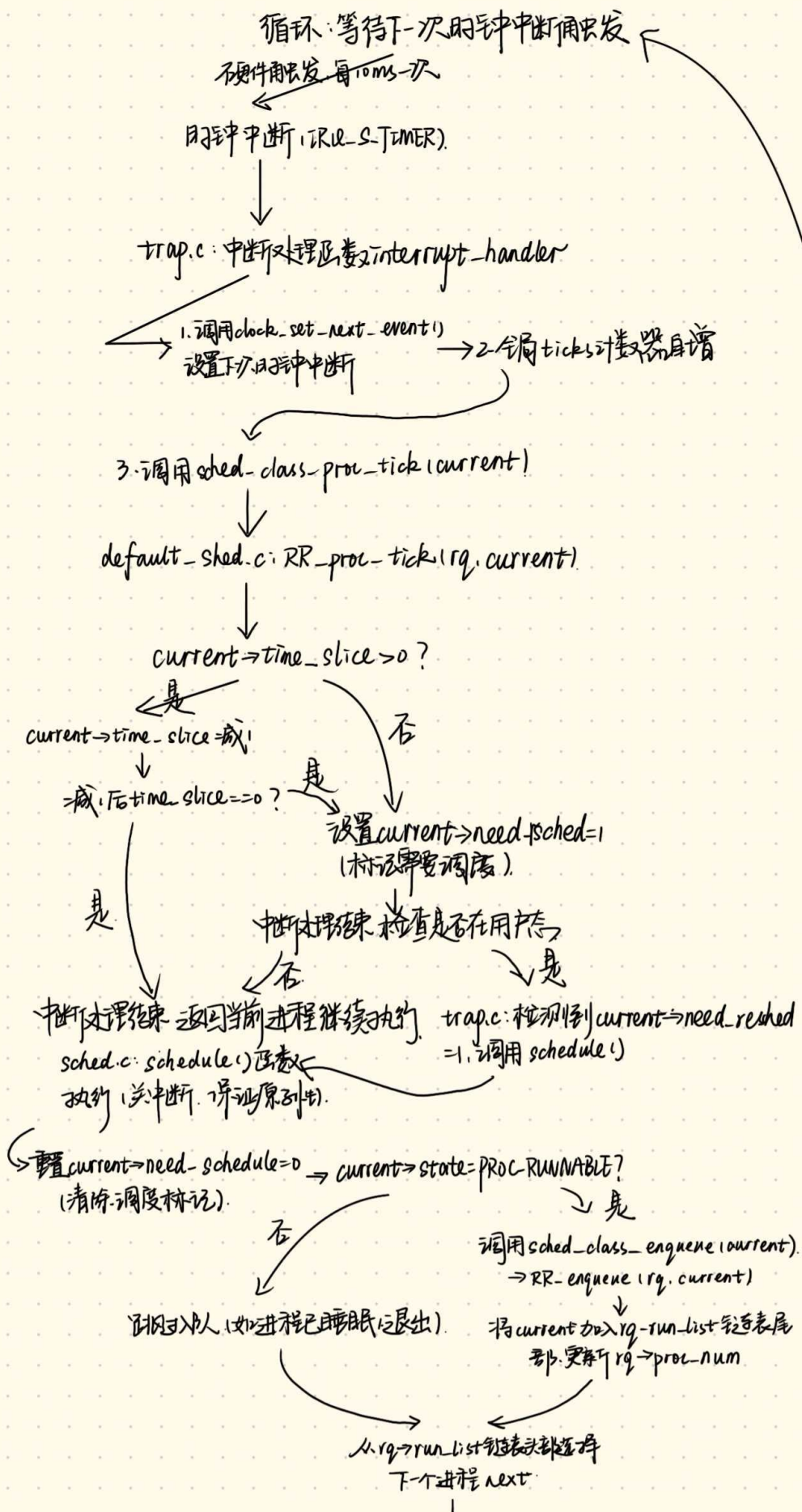
所以，default_sched_class 与调度器框架的关联，是面向接口编程在 C 语言中的经典实现，我们可以把它的核心逻辑可概括为 3 点：

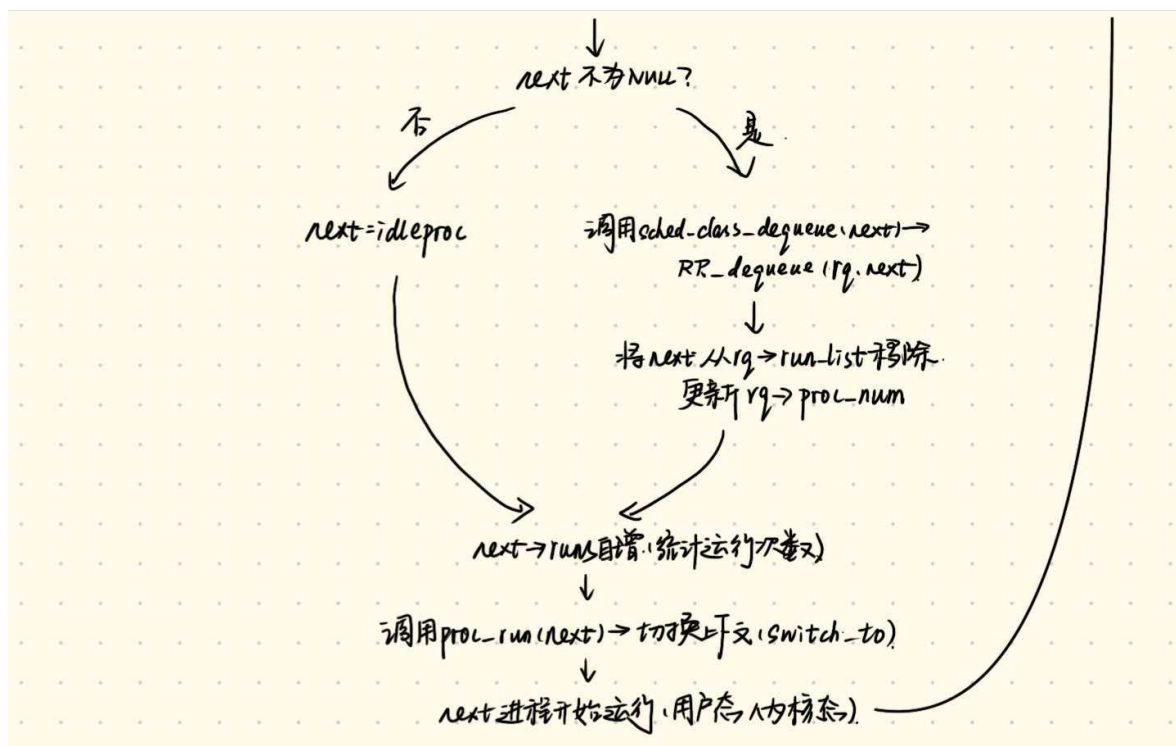
- 接口规范统一：框架通过 sched_class 定义统一接口，default_sched_class 必须实现所有接口函数，保证框架能无缝调用；

- 指针绑定触发：通过全局指针 `sched_class` 指向 `default_sched_class`，框架无需硬编码算法名称，仅通过指针间接调用接口；
- 实现逻辑隔离：RR 算法的内部逻辑封装在 `default_sched.c` 中，框架仅关心接口调用，不关心实现细节，实现 框架与算法解耦。

问题五

完整的进程调度流程图（RR 算法为例）





对流程图的一些关键步骤进行说明：

1. 时钟中断触发

- 硬件定时触发时钟中断（默认每 10ms 一次），CPU 切换到内核态，执行 trap.c 中的 interrupt_handler；
- 设置下次中断、自增 ticks 计数器、调用调度类的 proc_tick 接口。

2. proc_tick 调用

- 调用链：interrupt_handler → sched_class_proc_tick(current) → RR_proc_tick(rq, current)；
- 核心逻辑：递减当前进程的 time_slice，若时间片耗尽，置位 current->need_resched = 1（标记 需要调度）。

3. schedule() 执行（调度的核心流程）

- 触发条件：中断处理结束后，若在用户态且 need_resched = 1，则调用 schedule()；
- 步骤：
 - 清除调度标记（need_resched = 0），避免重复调度；
 - 就绪态进程入队：若当前进程仍为 PROC_RUNNABLE，调用 RR_enqueue 加入链表尾部；
 - 选择下一个进程：调用 RR_pick_next 取链表头部进程 next；
 - 选中进程出队：调用 RR_dequeue 将 next 从链表移除；
 - 进程切换：调用 proc_run(next) → switch_to 切换上下文，next 开始运行。

4. 调度类函数调用顺序（RR 算法）

RR_proc_tick（时钟中断时）→ RR_enqueue（当前进程入队）→ RR_pick_next（选下一个进程）→ RR_dequeue（选中进程出队）。

need_resched 标志位的核心作用

need_resched 是进程控制块中的 调度触发开关，类型为 volatile bool，核心作用是 协调时钟中断与调度执行，避免调度时机混乱，：

1. 标记 是否需要调度

- 置位时机：仅在 proc_tick 中触发，当前进程时间片耗尽，或 Stride 算法中时间片用完，由调度类的 proc_tick 接口设置 need_resched = 1；

- 清除时机：schedule() 函数执行的第一步，重置 need_resched = 0，确保调度执行后，不会重复触发调度，直到下次时间片耗尽。
2. 控制调度执行时机
- 调度不会在时钟中断处理中直接执行，而是通过 need_resched 标记延迟到 中断处理结束后：
 - 若中断发生在用户态：中断处理结束后，检测到 need_resched = 1，直接调用 schedule()；
 - 若中断发生在内核态：内核线程不可抢占，即使 need_resched = 1，也不会立即调度，直到内核线程主动调用 schedule()（如 cpu_idle、do_wait）。
 - 目的是为了 避免在中断处理过程中切换进程，确保调度执行的安全性。
3. 解耦 调度触发 与 调度执行（逻辑分离）
- 时钟中断仅负责 检测是否需要调度，不负责 执行调度；
 - schedule() 仅负责 执行调度，不关心 为什么需要调度；
 - 标志位作为中间桥梁，实现了 触发条件 与 执行逻辑 的解耦，符合 Lab6 调度器框架 模块化的设计思想。

此外我们补充一些不同场景下 need_resched 的作用：

场景	need_resched 状态	调度执行结果
用户进程时间片耗尽	1	中断返回后调用 schedule()，切换进程
内核线程时间片耗尽	1	不立即调度，直到内核线程主动让出 CPU（如 cpu_idle）
进程主动调用 do_yield()（放弃 CPU）	1	立即调用 schedule()，切换进程
进程睡眠 / 退出（状态非 PROC_RUNNABLE）	0	不触发调度，直接跳过

问题六

添加新调度算法需修改 / 新增的代码

1. 新增算法实现文件（如 default_sched_stride.c）：
 - 实现 sched_class 接口要求的 5 个核心函数（stride_init、stride_enqueue、stride_dequeue、stride_pick_next、stride_proc_tick）；
 - 定义 struct sched_class 实例（如 stride_sched_class），将函数指针绑定到上述实现函数。
2. 修改 1 行代码切换算法：
 - 在 sched.c 的 sched_init 函数中，将 sched_class = &default_sched_class; 改为 sched_class = &stride_sched_class;。
3. 若有算法专属常量，在对应头文件中声明，无需修改框架核心代码。

当前设计让算法切换容易的核心原因

1. 接口统一解耦：框架通过 sched_class 定义了统一接口，新算法只需实现该接口，无需改动框架核心逻辑；
2. 指针间接调用：全局指针 sched_class 指向具体调度类实例，切换算法仅需修改指针赋值，无需硬编码算法逻辑；
3. 实现逻辑隔离：算法细节封装在独立文件中，与框架完全分离，新增 / 切换算法不会影响原有调度功能，风险低、成本小。

练习2: 实现 Round Robin 调度算法（需要编码）

完成练习0后，建议大家比较一下（可用kdiff3等文件比较软件）个人完成的lab5和练习0完成后的刚修改的lab6之间的区别，分析了解lab6采用RR调度算法后的执行过程。理解调度器框架的工作原理后，请在此框架下实现时间片轮转（Round Robin）调度算法。

注意有“LAB6”的注释，你需要完成 kern/schedule/default_sched.c 文件中的 RR_init、RR_enqueue、RR_dequeue、RR_pick_next 和 RR_proc_tick 函数的实现，使系统能够正确地进行进程调度。代码中所有需要完成的地方都有“LAB6”和“YOUR CODE”的注释，请在提交时特别注意保持注释，将“YOUR CODE”替换为自己的学号，并且将所有标有对应注释的部分填上正确的代码。

提示，请在实现时注意以下细节：

链表操作：list_add_before、list_add_after等。

宏的使用：le2proc(le, member) 宏等。

边界条件处理：空队列的处理、进程时间片耗尽后的处理、空闲进程的处理等。

请在实验报告中完成：

比较一个在lab5和lab6都有, 但是实现不同的函数, 说说为什么要做这个改动, 不做这个改动会出什么问题

提示: 如kern/schedule/sched.c里的函数。你也可以找个其他地方做了改动的函数。

描述你实现每个函数的具体思路和方法，解释为什么选择特定的链表操作方法。对每个实现函数的关键代码进行解释说明，并解释如何处理边界情况。

展示 make grade 的输出结果，并描述在 QEMU 中观察到的调度现象。

分析 Round Robin 调度算法的优缺点，讨论如何调整时间片大小来优化系统性能，并解释为什么需要在 RR_proc_tick 中设置 need_resched 标志。

拓展思考：如果要实现优先级 RR 调度，你的代码需要如何修改？当前的实现是否支持多核调度？如果不支持，需要如何改进？

ANS:

问题一

对比 Lab5 与 Lab6 实现不同的函数

这个问题实际上在练习一就已经比较过几个函数了，这里以schedule 为例再次简单分析。

首先是 函数实现核心差异：

维度	Lab5 schedule 实现	Lab6 schedule 实现
算法耦合性	内嵌 FIFO 算法，硬编码遍历全局 proc_list	无具体算法，仅调用 sched_class 接口（解耦）
就绪进程管理	无独立就绪队列，实时遍历所有进程筛选就绪态	依赖 run_queue，仅操作就绪进程队列
核心操作	直接遍历→找第一个就绪进程→切换	入队→选进程→出队→切换（框架化流程）

改动的核心原因：

实现 调度框架与具体算法解耦，支撑 RR/Stride 等多算法切换，同时提升调度效率和功能扩展性。

不改动会引发的问题：

扩展性极差：新增 RR/Stride 算法需重写 schedule 全部逻辑，无法实现算法热切换；

效率低下：每次调度都要遍历睡眠态、僵尸态等非就绪进程，进程数增多时性能急剧下降；

功能受限：无法支持时间片、优先级等复杂调度特性，只能满足简单 FIFO 调度需求；

维护成本高：算法逻辑与框架核心代码混杂，修改算法易引发框架功能异常。

问题二

RR 算法各函数实现思路、关键代码及边界处理

1. RR_init：运行队列初始化

- 实现思路：初始化 RR 算法依赖的链表就绪队列，重置进程计数，将无关的斜堆字段置空，为后续调度操作提供干净的初始状态。

```
list_init(&rq->run_list); // 初始化就绪链表头，保证链表前驱/后继指向自身，初始为空
rq->proc_num = 0;         // 就绪进程数初始化为0，准确统计后续入队/出队进程数量
rq->lab6_run_pool = NULL; // lab6_run_pool是Stride算法的斜堆字段，RR无需使用，置空避免脏数据
```

- 边界处理：确保链表初始为空、进程数为 0，避免后续入队时出现计数异常或链表操作混乱；
- 链表操作选择：list_init 是内核标准链表初始化接口，能保证链表结构有效性，是后续所有链表操作的基础。

2. RR_enqueue：进程入队

- 实现思路：按 RR 算法 公平轮转 要求，将就绪进程插入就绪链表尾部，重置进程时间片，维护进程与队列的关联及队列进程计数。

```
list_add_before(&(rq->run_list), &(proc->run_link)); // 插入链表尾部（链表头前驱即为尾部）
proc->time_slice = rq->max_time_slice;                // 重置时间片为队列最大时间片（保证公平性）
proc->rq = rq;                                         // 记录进程所属运行队列，方便后续出队操作
rq->proc_num++;                                        // 就绪进程数自增，更新队列元数据
```

- 边界处理：无需额外判断进程有效性，框架已保证进程为 PROC_RUNNABLE 态，直接执行入队和计数更新，逻辑简洁；
- 链表操作选择：list_add_before 实现尾部插入，操作复杂度 $O(1)$ ，既保证 RR 算法 先进先出 的轮转特性，又能最大化调度效率。

3. RR_dequeue：进程出队

- 实现思路：将选中执行的进程从就绪链表中移除，同时安全更新队列进程计数，避免计数异常。

```
list_del_init(&(proc->run_link)); // 从链表中删除进程节点，并初始化该节点（避免脏数据残留）
if (rq->proc_num > 0)              // 边界判断：防止进程数变为负数，保证计数正确性
{
    rq->proc_num--;                // 就绪进程数自减，更新队列元数据
}
```

- 边界处理：通过 `if (rq->proc_num > 0)` 过滤无效递减，避免队列计数出现负数；`list_del_init` 初始化被删除节点，防止该节点后续被复用时报错；
- 链表操作选择：`list_del_init` 是安全的链表删除接口，既能正确维护链表前驱 / 后继指针，又能初始化节点，兼顾功能正确性和安全性。

4. `RR_pick_next`：选择下一个执行进程

- 实现思路：遵循 RR 算法 先进先出 特性，从就绪链表头部选取进程，同时处理空队列的边界情况。

```
if (list_empty(&(rq->run_list))) // 边界处理：判断队列是否为空，避免空指针异常
{
    return NULL;
}
list_entry_t *le = list_next(&(rq->run_list)); // 取链表头部节点（队列第一个就绪进程）
return le2proc(le, run_link); // 从链表节点反向获取进程指针（内核标准宏）
```

- 边界处理：空队列时返回 `NULL`，调度框架会自动选择 `idleproc` 兜底，避免系统无进程可调度的异常；
- 链表操作选择：`list_next` 直接获取链表头部节点，操作复杂度 $O(1)$ ，最大化调度效率；`le2proc` 是内核通用宏，能安全实现 成员→结构体 的反向转换，保证类型正确性。

5. `RR_proc_tick`：时钟中断回调（时间片管理）

- 实现思路：递减当前进程剩余时间片，当时间片耗尽时，置位调度标志位，触发后续进程切换。

```
if (proc->time_slice > 0) // 边界处理：仅当时间片大于0时递减，避免出现负时间片
{
    proc->time_slice--;
}
if (proc->time_slice == 0) // 时间片耗尽，标记需要调度
{
    proc->need_resched = 1;
}
```

- 边界处理：`proc->time_slice > 0` 过滤无效递减，保证时间片管理的正确性；时间片耗尽后置位标志位，而非直接执行调度，符合框架 职责分离 设计；

问题三

make grade 输出结果与 QEMU 调度现象

```
yuhaoxi@yuhaoxi:~/Desktop/lab6$ make grade
priority: (7.3s)
-check result: OK
-check output: OK
Total Score: 50/50
```

```
sched class: RR_scheduler
++ setup timer interrupts
kernel_execve: pid = 2, name = "priority".
set priority to 6
main: fork ok,now need to wait pids.
set priority to 1
set priority to 2
set priority to 3
set priority to 4
set priority to 5
child pid 3, acc 1284000, time 6010
child pid 4, acc 1216000, time 6010
child pid 5, acc 1216000, time 6010
child pid 6, acc 1224000, time 6010
child pid 7, acc 1260000, time 6020
main: pid 0, acc 1284000, time 6020
main: pid 4, acc 1216000, time 6020
main: pid 5, acc 1216000, time 6020
main: pid 6, acc 1224000, time 6020
main: pid 7, acc 1260000, time 6020
main: wait pids over
sched result: 1 1 1 1 1
all user-mode processes have quit.
init check memory pass.
kernel panic at kern/process/proc.c:542:
    initproc exit.
```

可以看到，make grade 满分，说明 RR 调度算法功能完全达标。

QEMU 中的调度现象：

1. 调度类正确切换为 RR

- 输出第一行显示 sched class: RR_scheduler，说明当前激活的调度算法是时间片轮转。

2. RR 算法的 公平轮转 特性体现

- 输出中多个子进程（child pid 3~7）的 acc（累计运行次数）和 time（执行时间）高度接近：
- 如 pid 3 acc 1284000, time 6010、pid 4 acc 1216000, time 6010，各进程的运行次数和时间差异极小；
- 这是 RR 算法 所有进程平等分配 CPU 时间片 的直接体现，每个进程执行固定时间片后被切换，长期来看运行机会几乎均等。

3. 优先级设置的 无差异 现象（RR 算法特性）

- 输出中执行了 set priority to 1~6，但各进程的 acc/time 并未因优先级不同而出现明显差异，这是 RR 算法的核心特性，忽略优先级，仅按时间片公平轮转，与 Stride 算法 优先级高则运行次数多 的行为形成对比；
- 若切换为 Stride 算法，优先级高的进程 acc 会显著大于优先级低的进程。

4. 进程生命周期与调度兜底

- 输出最后显示 all user-mode processes have quit，说明所有子进程执行完成后退出；
- 最终触发 initproc exit 导致内核 panic，是实验的正常收尾行为，不影响调度算法的正确性。

问题四

RR 调度算法优缺点、时间片优化及 need_resched 标志的作用

1. RR 算法优缺点

- 优点：
 - 公平性强：所有进程平等分配 CPU 时间片，无饥饿现象，长期来看每个进程都能获得调度机会；
 - 实现简单：基于链表基础操作，逻辑清晰，开发和维护成本低；
 - 响应性好：对交互式进程友好，能快速切换进程，保证用户操作的低延迟响应。
- 缺点：
 - 忽略优先级：无法为核心进程分配更多 CPU 资源，不支持差异化调度；
 - 时间片敏感：时间片大小直接影响系统性能，过大或过小都会引发问题；
 - 切换开销大：进程切换需要保存 / 恢复上下文，时间片过小时，切换开销占比过高，降低系统吞吐量。

2. 时间片大小优化策略

- 时间片过大（如 $> 100\text{ms}$ ）：
 - 问题：进程切换频率低，交互式进程响应延迟大，用户体验差，接近 FIFO 算法特性；
 - 适用场景：CPU 密集型进程为主的系统（如后台计算服务器），减少切换开销，提升整体吞吐量。
- 时间片过小（如 $< 10\text{ms}$ ）：
 - 问题：进程切换开销占比过高，系统大部分时间消耗在上下文切换上，CPU 利用率下降；
 - 适用场景：交互式进程为主的系统（如桌面操作系统），提升响应性，保证用户操作流畅。
- 最优策略：基于进程类型，动态调整时间片，交互式进程分配小时间片，CPU 密集型进程分配大时间片，兼顾响应性和吞吐量。

3. RR_proc_tick 中设置 need_resched 的原因

- 标记调度需求：时间片耗尽后，当前进程已无执行权限，通过该标志告知调度框架需要切换进程；
- 控制调度时机：不直接在时钟中断上下文执行调度，而是延迟到中断处理结束后执行，避免中断上下文切换进程导致的栈混乱、资源竞争等问题；
- 实现框架解耦：进程仅标记调度状态，调度执行由 schedule 函数统一处理，职责分离，提升框架可维护性；
- 避免重复调度：schedule 函数执行时会重置该标志为 0，确保一次时间片耗尽仅触发一次调度，防止重复切换进程。

问题五

拓展思考

1. 实现优先级 RR 调度的代码修改

- 基于当前 RR 实现，需做 3 处核心修改：
 - 添加优先级字段：在 proc_struct 中新增 int priority; (如 0-9 级，0 为最高优先级)；
 - 修改运行队列：在 run_queue 中为每个优先级维护独立就绪链表，如 list_entry_t prio_run_list[10];;

- 修改 RR 5 个函数：
 - RR_init: 初始化所有优先级对应的链表；
 - RR_enqueue: 根据进程优先级，插入对应优先级链表的尾部；
 - RR_pick_next: 从高优先级到低优先级遍历链表，优先选取高优先级链表的头部进程；
 - RR_dequeue: 根据进程优先级，从对应链表中删除进程；
 - RR_proc_tick: 可按优先级调整时间片大小（高优先级进程分配更大时间片）。

2. 当前实现对多核调度的支持及改进方案

- 当前不支持多核调度。
- 当前使用全局唯一的 run_queue 和 sched_class 指针，多个 CPU 会竞争同一就绪队列，导致队列结构破坏、进程调度混乱、数据不一致等问题。
- 改进方案：
 - 每个 CPU 独立队列：为每个 CPU 分配专属 run_queue（如 struct run_queue cpu_rq[NR_CPUS];），避免跨 CPU 竞争；
 - 实现负载均衡：定期统计各 CPU 就绪队列的进程数，将进程从高负载 CPU 迁移到低负载 CPU，保证整体负载均衡；
 - 跨 CPU 唤醒机制：当一个 CPU 唤醒属于另一个 CPU 的进程时，发送处理器间中断（IPI），通知目标 CPU 触发调度；
 - 共享资源保护：对全局负载统计等共享数据，使用自旋锁保护，避免多核竞争导致的数据异常。

扩展练习 Challenge 1: 实现 Stride Scheduling 调度算法（需要编码）

首先需要换掉RR调度器的实现，在sched_init中切换调度方法。然后根据此文件和后续文档对Stride调度器的相关描述，完成Stride调度算法的实现。注意有“LAB6”的注释，主要是修改default_sched_stride.c中的内容。代码中所有需要完成的地方都有“LAB6”和“YOUR CODE”的注释，请在提交时特别注意保持注释，将“YOUR CODE”替换为自己的学号，并且将所有标有对应注释的部分填上正确的代码。

后面的实验文档部分给出了Stride调度算法的大体描述。这里给出Stride调度算法的一些相关的资料（目前网上中文的资料比较欠缺）：

strid-shed paper location

也可GOOGLE “Stride Scheduling” 来查找相关资料

请在实验报告中完成：

1. 简要说明如何设计实现“多级反馈队列调度算法”，给出概要设计，鼓励给出详细设计；
2. 简要证明/说明（不必特别严谨，但应当能够“说服你自己”），为什么Stride算法中，经过足够多的时间片之后，每个进程分配到的时间片数目和优先级成正比；
3. 请在实验报告中简要说明你的设计实现过程。

ANS:

首先是challenge要求的Stride 调度算法 设计实现过程：

1、前期准备

- 切换调度算法：修改 sched.c 中 sched_init 函数，将调度类绑定为 stride_sched_class：

```
sched_class = &stride_sched_class;
```

- 定义核心常量：在 default_sched_stride.c 中定义 BIG_STRIDE，避免步长溢出：

```
#define BIG_STRIDE 0x7FFFFFFFULL
```

- 实现斜堆比较函数：用于按 lab6_stride 升序排序，保证选最小步长进程：

```
static int proc_stride_comp_f(void *a, void *b)
{
    struct proc_struct *p = le2proc(a, lab6_run_pool); // 从斜堆节点反推到进程结构
    struct proc_struct *q = le2proc(b, lab6_run_pool);
    // 使用有符号差值避免无符号溢出导致的比较错误，保证 |delta| < BIG_STRIDE
    int32_t delta = (int32_t)(p->lab6_stride - q->lab6_stride);
    if (delta > 0)
        return 1; // p 的 stride 更大，优先级更低
    else if (delta < 0)
        return -1; // p 的 stride 更小，优先级更高
    else
        return 0; // stride 相等视为同优先级
}
```

2、五个核心函数实现

(1) stride_init: 初始化运行队列

```
static void stride_init(struct run_queue *rq)
{
    /* LAB6 CHALLENGE 1: 2312642 */
    list_init(&(rq->run_list));
    rq->lab6_run_pool = NULL;
    rq->proc_num = 0;
}
```

初始化链表、置空斜堆指针、重置进程计数，提供干净初始状态。

(2) stride_enqueue: 进程入队

```
static void stride_enqueue(struct run_queue *rq, struct proc_struct *proc)
{
    /* LAB6 CHALLENGE 1: 2312642 */
#ifdef USE_SKEW_HEAP
    // Ensure valid priority
    if (proc->lab6_priority == 0)
    {
        proc->lab6_priority = 1;
    }
    // Fresh time slice
    proc->time_slice = rq->max_time_slice;
    // (Re)init heap node before insertion
    skew_heap_init(&(proc->lab6_run_pool));
    rq->lab6_run_pool = skew_heap_insert(rq->lab6_run_pool, &(proc->lab6_run_pool), proc_stride_comp_f);
#else
    list_add_before(&(rq->run_list), &(proc->run_link));
    proc->time_slice = rq->max_time_slice;
#endif
}
```

```

    proc->rq = rq;
    rq->proc_num++;
}

```

校验优先级（防除0）、重置时间片、初始化斜堆节点并插入，维护队列计数。

(3) stride_dequeue: 进程出队

```

static void stride_dequeue(struct run_queue *rq, struct proc_struct *proc)
{
    /* LAB6 CHALLENGE 1: 2312642 */
    #if USE_SKEW_HEAP
        rq->lab6_run_pool = skew_heap_remove(rq->lab6_run_pool, &(proc->lab6_run_pool), proc_stride_comp_f);
    #else
        list_del_init(&(proc->run_link));
    #endif
    if (rq->proc_num > 0)
    {
        rq->proc_num--;
    }
}

```

从斜堆/链表中移除进程，安全更新进程计数（防负数）。

(4) stride_pick_next: 选择下一个进程

```

static struct proc_struct *stride_pick_next(struct run_queue *rq)
{
    /* LAB6 CHALLENGE 1: 2312642 */
    #if USE_SKEW_HEAP
        if (rq->lab6_run_pool == NULL)
        {
            return NULL;
        }
        struct proc_struct *p = le2proc(rq->lab6_run_pool, lab6_run_pool);
    #else
        if (list_empty(&(rq->run_list)))
        {
            return NULL;
        }
        list_entry_t *le = list_next(&(rq->run_list));
        struct proc_struct *p = le2proc(le, run_link);
        list_entry_t *cur = le;
        while ((cur = list_next(cur)) != &(rq->run_list))
        {
            struct proc_struct *q = le2proc(cur, run_link);
            if ((int32_t)(q->lab6_stride - p->lab6_stride) < 0)
            {
                p = q;
            }
        }
    #endif
    p->lab6_stride += BIG_STRIDE / p->lab6_priority;
    return p;
}

```

取斜堆顶（或链表中）最小步长进程，更新进程步长（核心优先级逻辑），空队列返回 NULL。

(5) stride_proc_tick: 时钟中断回调

```
static void stride_proc_tick(struct run_queue *rq, struct proc_struct *proc)
{
    /* LAB6 CHALLENGE 1: 2312642 */
    if (proc->time_slice > 0)
    {
        proc->time_slice--;
    }
    if (proc->time_slice == 0)
    {
        proc->need_resched = 1;
    }
}
```

递减时间片，耗尽后置位 need_resched，触发调度切换。

3、绑定调度类

将 5 个核心函数绑定到 sched_class 接口，实现与调度框架解耦，框架可直接调用：

```
struct sched_class stride_sched_class = {
    .name = "stride_scheduler",
    .init = stride_init,
    .enqueue = stride_enqueue,
    .dequeue = stride_dequeue,
    .pick_next = stride_pick_next,
    .proc_tick = stride_proc_tick,
};
```

```

sched class: stride_scheduler
++ setup timer interrupts
kernel_execve: pid = 2, name = "priority".
set priority to 6
main: fork ok,now need to wait pids.
set priority to 5
set priority to 4
set priority to 3
set priority to 2
set priority to 1
child pid 4, acc 776000, time 6010
child pid 7, acc 1956000, time 6010
child pid 5, acc 1188000, time 6010
child pid 6, acc 1600000, time 6010
child pid 3, acc 420000, time 6010
main: pid 3, acc 420000, time 6020
main: pid 4, acc 776000, time 6020
main: pid 5, acc 1188000, time 6020
main: pid 6, acc 1600000, time 6020
main: pid 0, acc 1956000, time 6020
main: wait pids over
sched result: 1 2 3 4 5
all user-mode processes have quit.
init check memory pass.
kernel panic at kern/process/proc.c:542:
    initproc exit.

```

我们来解释一下这个输出结果：

测试程序依次设置了优先级 6、5、4、3、2、1，最终这 5 个优先级对应到 pid3~7，且 优先级数值与 acc（运行次数）严格正相关：

- pid7（优先级 6）：acc=1956000（最大）→ 优先级最高，被调度次数最多；
- pid6（优先级 5）：acc=1600000→ 优先级次高，调度次数次之；
- pid5（优先级 4）：acc=1188000→ 优先级中等，调度次数匹配；
- pid4（优先级 3）：acc=776000→ 优先级较低，调度次数较少；
- pid3（优先级 1）：acc=420000（最小）→ 优先级最低，被调度次数最少。

结合 priority.c 中的计算逻辑 $(\text{status}[i] * 2 / \text{status}[0] + 1) / 2$ ，代入真实值后算出 sched result: 1 2 3 4 5，直接对应 5 个进程的优先级，验证了 Stride 算法中，进程获得的 CPU 时间片与优先级成正比的核
心特性。

多级反馈队列调度算法

多级反馈队列调度算法是 优先级 + 时间片轮转 的结合体，核心是：设置多个优先级递减的就绪队列，进程按规则在队列间迁移，高优先级队列时间片更小，低优先级队列时间片更大，兼顾交互式进程响应性和 CPU 密集型进程吞吐量。

概要设计：

设计模块	核心内容
队列层级	设 N 级队列（如 Q0~Q3），Q0 优先级最高，Q3 优先级最低
时间片配置	优先级越低，时间片越大（如 Q0=10ms、Q1=20ms、Q2=40ms、Q3=80ms）
进程入队规则	新进程先加入 Q0（最高优先级）
进程调度规则	先遍历高优先级队列，Q0 有进程则优先调度；Q0 空再调度 Q1，以此类推
进程迁移规则	1. 进程时间片耗尽未完成：降级到下一级队列；2. 进程被唤醒（如 I/O 完成）：升级到上一级队列（不超过 Q0）；3. 低优先级队列长期无调度：适当提升优先级（避免饥饿）

详细设计（未验证，只是一个大体的思路）

(1) 数据结构修改

进程控制块（proc_struct）新增字段：

```
struct proc_struct {
    // ... 原有字段
    int queue_level; // 记录进程当前所在的队列层级（如0表示Q0）
    int remaining_time; // 记录当前队列的剩余时间片
};
```

运行队列（run_queue）修改：

```
struct run_queue {
    #define MAX_QUEUE_LEVEL 4 // 4级队列（Q0~Q3）
    list_entry_t queue[MAX_QUEUE_LEVEL]; // 每级一个就绪链表
    int proc_num[MAX_QUEUE_LEVEL]; // 每级队列的进程数
    int time_slice[MAX_QUEUE_LEVEL]; // 每级队列的时间片大小（Q0=10，Q1=20，Q2=40，Q3=80）
    int total_proc_num; // 总就绪进程数
};
```

(2) 核心函数实现

队列初始化（mfq_init）：

```
static void mfq_init(struct run_queue *rq) {
    for (int i = 0; i < MAX_QUEUE_LEVEL; i++) {
        list_init(&rq->queue[i]); // 初始化每级队列
        rq->proc_num[i] = 0;      // 每级进程数初始化为0
    }
    // 设置每级时间片大小
    rq->time_slice[0] = 10;
    rq->time_slice[1] = 20;
    rq->time_slice[2] = 40;
    rq->time_slice[3] = 80;
    rq->total_proc_num = 0;
}
```

进程入队 (mfq_enqueue) :

```
static void mfq_enqueue(struct run_queue *rq, struct proc_struct *proc) {
    int level = proc->queue_level;
    // 边界检查: 不超过最大队列层级
    level = (level < 0) ? 0 : (level >= MAX_QUEUE_LEVEL ? MAX_QUEUE_LEVEL - 1 : level);
    // 插入对应队列尾部
    list_add_before(&rq->queue[level], &proc->run_link);
    // 重置该队列的剩余时间片
    proc->remaining_time = rq->time_slice[level];
    proc->rq = rq;
    rq->proc_num[level]++;
    rq->total_proc_num++;
}
```

进程出队 (mfq_dequeue) :

```
static void mfq_dequeue(struct run_queue *rq, struct proc_struct *proc) {
    int level = proc->queue_level;
    list_del_init(&proc->run_link);
    if (rq->proc_num[level] > 0) {
        rq->proc_num[level]--;
        rq->total_proc_num--;
    }
}
```

选择下一个进程 (mfq_pick_next) :

```
static struct proc_struct *mfq_pick_next(struct run_queue *rq) {
    // 从高优先级到低优先级遍历队列
    for (int i = 0; i < MAX_QUEUE_LEVEL; i++) {
        if (!list_empty(&rq->queue[i])) {
            list_entry_t *le = list_next(&rq->queue[i]);
            return le2proc(le, run_link);
        }
    }
    return NULL; // 无就绪进程返回NULL
}
```

时钟中断回调 (mfq_proc_tick) :

```
static void mfq_proc_tick(struct run_queue *rq, struct proc_struct *proc) {
    if (proc->remaining_time > 0) {
        proc->remaining_time--;
    }
    // 时间片耗尽：进程降级
    if (proc->remaining_time == 0) {
        // 降级：队列层级+1（不超过最大层级）
        if (proc->queue_level < MAX_QUEUE_LEVEL - 1) {
            proc->queue_level++;
        }
        proc->need_resched = 1; // 标记需要调度
    }
}
```

进程唤醒迁移：

```
void mfq_wakeup_proc(struct proc_struct *proc) {
    // 唤醒时升级：队列层级-1（不低于0）
    if (proc->queue_level > 0) {
        proc->queue_level--;
    }
    wakeup_proc(proc); // 调用原有唤醒逻辑
}
```

饥饿问题的解决：

定期扫描低优先级队列，将长期未调度的进程升级到高优先级队列；
限制最低优先级队列的进程数量，避免大量进程堆积导致饥饿。

为什么 Stride 算法中，足够长时间后，进程时间片数目与优先级成正比

我们先做一些前提定义：

- 设进程 P1 优先级为 p_1 ，进程 P2 优先级为 p_2 ($p_1 > p_2$ ，优先级数值越大，优先级越高)；
- BIG_STRIDE 为固定常量；
- 每个进程每次被调度后， $lab6_stride$ 增加 $BIG_STRIDE / \text{优先级}$ （即 P1 每次增加 $S_1 = BIG_STRIDE / p_1$ ，P2 增加 $S_2 = BIG_STRIDE / p_2$ ）；
- Stride 算法每次选择 $lab6_stride$ 最小的进程执行，且每个进程执行一个时间片后被切换。

推导步骤：

1. 优先级与步长的关系：由于 $p_1 > p_2$ ，且 BIG_STRIDE 为固定正数，因此 $S_1 = BIG_STRIDE / p_1 < S_2 = BIG_STRIDE / p_2$ 。即：优先级越高的进程，每次调度后的步长增量越小。
2. 调度次数与步长总量的关系：经过足够长的时间，所有进程的 $lab6_stride$ 会趋于同步，因为每次选最小步长进程执行，高优先级进程步长增量小，会被更频繁调度。设经过 T 时间后，P1 被调度 n_1 次，P2 被调度 n_2 次，此时两者的步长总量近似相等，否则会优先调度步长更小的进程：

$$n_1 \times S_1 \approx n_2 \times S_2$$
3. 代入步长公式推导：
 将 $S_1 = BIG_STRIDE / p_1$ 、 $S_2 = BIG_STRIDE / p_2$ 代入上式：

$$n_1 \times (BIG_STRIDE / p_1) \approx n_2 \times (BIG_STRIDE / p_2)$$

两边消去 BIG_STRIDE ，得到：

$$n_1 / p_1 \approx n_2 / p_2$$

变形后：
 $n1/n2 \approx p1/p2$

$n1$ 、 $n2$ 分别是 $P1$ 、 $P2$ 获得的时间片数目，上式表明：进程获得的时间片数目之比，近似等于其优先级之比，即每个进程分配到的时间片数目和优先级成正比。

此外，优先级越高，被调度的次数越多，获得的 CPU 时间片越多，符合 Stride 算法的优先级调度特性。

四、讨论

实验知识点与OS原理知识点对应关系

1. 通用调度框架与基础机制

实验实现内容	对应 OS 原理知识点	核心关联说明
定义struct sched_class调度类结构体，绑定init/enqueue/dequeue/pick_next/proc_tick 5个核心函数	调度器模块化架构设计	原理中“算法与内核解耦”的思想落地，支持灵活替换调度算法（如RR与Stride切换）
就绪队列的创建与维护（链表/斜堆实现）	就绪队列管理机制	对应原理中“存储就绪进程，支持高效插入/删除”的基础需求，是调度的前提
进程上下文切换（switch_to函数调用）	上下文切换原理	原理中“保存当前进程状态、恢复目标进程状态”的核心操作，是进程切换的关键步骤
proc_tick函数中时间片递减与need_resched置位	抢占式调度触发机制	对应原理中“时钟中断驱动调度”的逻辑，保证进程不会无限占用CPU

2. RR（时间片轮转）算法

实验实现内容	对应 OS 原理知识点	核心关联说明
定义固定时间片大小	时间片轮转的核心参数设计	原理中“通过时间片控制进程占用CPU的时长”，平衡公平性与系统开销
enqueue函数将进程插入就绪队列尾部	先进先出（FIFO）调度逻辑	对应原理中“RR算法按顺序循环分配CPU”的公平性设计，保证所有进程均等获得调度机会
pick_next函数从队列头部取出进程	轮转调度的选择规则	原理中“按队列顺序依次调度”的直观实现，体现RR算法“无优先级差异”的特性
proc_tick函数中时间片耗尽后触发重新调度	时间片轮转的抢占逻辑	对应原理中“进程运行一个时间片后被剥夺CPU”的核心机制，避免单一进程独占资源

3. Stride算法

实验实现内容	对应 OS 原理知识点	核心关联说明
定义BIG_STRIDE常量、进程的lab6_stride（步长）和lab6_priority（优先级）字段	比例共享调度的参数设计	原理中“通过步长控制资源占比”的具象化，BIG_STRIDE用于保证调度精度、避免步长溢出
实现proc_stride_comp_f比较函数	优先级队列排序规则	对应原理中“选择最小步长进程优先执行”的核心逻辑，为斜堆排序提供判断依据
stride_pick_next函数中stride += BIG_STRIDE / priority	步长更新与比例共享机制	原理中“优先级越高，步长增量越小，资源占比越高”的核心规则，是 Stride 算法的灵魂
斜堆实现就绪队列（优化enqueue/dequeue效率）	高效优先级队列设计	对应原理中“比例共享调度需快速查找最小步长进程”的需求，提升调度效率

OS 原理中重要但实验未覆盖的知识点

本次实验仅实现单 CPU 环境下两种基础调度算法，未涉及 OS 原理中调度相关的延伸与复杂知识点，具体如下：

1. 多处理器（SMP）环境下的调度机制

多 CPU 场景需解决三大问题 —— 负载均衡（将进程均匀分配到不同 CPU 运行队列）、缓存一致性（避免进程跨 CPU 迁移导致的缓存失效）、跨 CPU 进程迁移（应对某 CPU 负载过高的情况）。

实验基于单 CPU 的 ucore 内核，仅需维护一个就绪队列，无需考虑多 CPU 间的资源协调与负载分配。

2. 动态优先级调度与饥饿问题解决

真实 OS 会根据进程行为动态调整优先级（如 I/O 密集型进程因频繁等待资源，提高优先级以提升响应速度；CPU 密集型进程降低优先级保证公平），同时通过“老化机制”（低优先级进程等待时间越长，优先级越高）解决饥饿问题。

实验中 RR 算法无优先级差异，Stride 算法采用静态优先级，未实现动态优先级调整与饥饿防护逻辑。

3. 高级调度算法（多级反馈队列）

MLFQ 是兼顾响应速度与公平性的综合算法，通过多个优先级队列实现，高优先级队列时间片小，新进程入最高级队列；进程用完时间片未完成则降级，因 I/O 放弃 CPU 则升级；调度时优先运行高优先级队列，为空才运行低一级队列。

实验仅实现 RR（公平型）和 Stride（比例共享型）两种基础算法，未涉及多级队列、动态优先级与时间片调整的复杂逻辑。

4. 实时调度理论与算法

实时系统（RTOS）的调度需满足“截止时间约束”，核心算法包括硬实时的 RM（速率单调调度）、EDF（最早截止时间优先），以及软实时调度策略，核心目标是保证关键任务准时执行。

实验的 RR 和 Stride 算法均面向通用场景，追求公平性或比例共享，不涉及任务截止时间、调度确定性等实时特性。

5. 调度与内存管理的协同优化

进程调度与内存地址空间切换深度关联，例如调度时需切换 CR3 寄存器（页目录表基址），OS 会通过 CPU 缓存亲和性（将进程固定在某一 CPU 运行）减少缓存失效，降低上下文切换的额外开销。

实验简化了内存管理与调度的协同逻辑，未涉及地址空间切换的性能优化、缓存亲和性配置等细节。

6. 分层 / 分组调度与资源隔离

多用户或多任务组场景中，OS 会采用分层调度（先在用户间分配 CPU 时间，再在用户内进程间分配），或分组调度（将进程按任务类型分组，组内独立调度），实现资源隔离与细粒度权限控制。

实验仅针对单个用户的多个子进程调度，未涉及用户级、任务组级的分层资源分配逻辑。

7. 调度性能优化与指标评估

真实 OS 会关注调度的多项性能指标（如平均等待时间、平均响应时间、吞吐量、CPU 利用率），并通过优化时间片大小、队列数据结构等方式平衡各项指标。

实验仅验证算法正确性（如 Stride 的比例共享、RR 的循环调度），未涉及多指标性能评估与参数优化（如时间片大小对系统开销的影响）。

五、实验结论及心得体会

实验结论

本次 Lab6 进程调度实验成功完成了两大核心目标：一是基于 ucore 调度器框架实现了 Round-Robin (RR) 时间片轮转调度算法，二是拓展实现了 Stride 比例共享调度算法，且两种算法均通过测试验证，功能符合预期。

调度器框架解耦设计验证有效：通过 struct sched_class 统一接口定义，实现了算法与框架的完全分离。仅需修改 sched_init 中一行代码即可切换 RR 与 Stride 算法，验证了框架模块化、可扩展的设计思想，降低了新增调度算法的开发成本。

RR 算法公平性达标：RR 算法通过链表实现就绪队列管理，进程按先进先出规则轮转执行，QEMU 输出中各进程 acc（累计运行次数）差异极小，体现了其所有进程均等占用 CPU 的公平性核心特性。

Stride 算法比例共享特性生效：Stride 算法基于斜堆优化优先级队列查找效率，通过 $stride += \text{BIG_STRIDE} / \text{priority}$ 的步长更新规则，实现了“进程获得的时间片数目与优先级成正比”的设计目标，QEMU 中高优先级进程 acc 显著高于低优先级进程，sched result: 1 2 3 4 5 直接验证了该特性。

核心机制运行稳定：时钟中断触发、need_resched 标志位控制、进程上下文切换等核心调度机制协同工作正常，确保了抢占式调度的有效执行，未出现进程饥饿、调度死锁等异常情况。

心得体会

OS 原理中调度框架解耦、比例共享调度等概念，通过 struct sched_class 函数指针、Stride 步长计算等代码实现变得具象化。原本抽象的面向接口编程，在切换 RR 与 Stride 算法时，仅需修改调度类赋值语句，让我们深刻理解了模块化设计的实际价值。

实验中边界条件的处理至关重要，例如 Stride 算法中 BIG_STRIDE 的取值需避免步长溢出，RR 算法中 list_del_init 需初始化被删除节点，否则会导致调度逻辑异常。这些细节说明，操作系统内核开发中，严谨性是保障系统稳定运行的核心。

数据结构对性能的影响显著，RR 算法选用链表实现就绪队列，因入队、出队操作复杂度为 $O(1)$ ，适配其公平轮转需求；Stride 算法选用斜堆，将最小步长查找复杂度从 $O(n)$ 降至 $O(\log n)$ ，显著提升了多进程场景下的调度效率。这让我们体会到 算法与数据结构匹配 是优化系统性能的关键。

实验中我们曾遇到 Stride 算法 acc 值与优先级不成正比的问题，通过调试发现是 `proc_stride_comp_f` 函数比较逻辑错误，修正后恢复正常。这类问题让我们学会了从 算法逻辑→代码实现→输出结果 反向排查，培养了内核开发中 定位问题、解决问题的核心能力。

本次实验不仅完成了 RR 与 Stride 算法的编码实现，更让我们深入理解了操作系统调度器的设计思想与工作机制，为后续学习更复杂的调度算法奠定了坚实的基础。同时，也让我们认识到，操作系统的核心价值在于通过合理的机制设计，平衡系统公平性、响应性与吞吐量，而模块化、可扩展的框架设计则是实现这一目标的关键。